

Data-Driven Fleet Operations: Demand Analytics and Smart Charging for Urban Ride-Hailing in Chicago

Advanced Analytics and Applications

Team 06



Authors:

Louis Schreier (7381065)

Daniel Schumacher (7409287)

Timo Ercole (7408459)

Contact: student@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

1	Introduction	1
1.1	Business Context	1
1.2	Business Goal and Data Mining Goal	1
2	Data Description	1
2.1	Taxi Trip Data	1
2.2	Weather Data	2
2.3	Point-of-Interest Data	3
3	Data Preparation	4
3.1	Taxi Data Cleaning	4
3.1.1	Consistency Checks	4
3.1.2	Billing Discrepancy	4
3.1.3	Outlier Removal	5
3.1.4	Two Cleaning Levels	5
3.2	Weather Data Preparation	5
3.3	Point-of-Interest Data Preparation	6
3.4	Feature Engineering	6
3.4.1	Spatial and Temporal Granularity	6
3.4.2	Features Used in the Forecasting Models	6
4	Descriptive Analytics	7
4.1	Temporal and Trip Patterns	7
4.2	Spatial Analytics	7
4.2.1	Spatial Discretization: H3 Hexagonal Grid	7
4.2.2	Trip Metric Distributions	8
4.2.3	Demand by Community Area and Hexagon Resolution	8
4.2.4	Temporal Demand Patterns Across Space	9
4.2.5	Demand Across the Weekly Cycle	10
4.2.6	Origin–Destination Analysis	11
4.2.7	Spatio-Temporal Net Flow Analysis	12
5	Data Analytics	13
5.1	Predictive Analytics	13
5.1.1	Problem Formulation and Validation Strategy	13
5.1.2	Support Vector Machine	13
5.1.3	Feedforward Neural Network	15
5.1.4	Model Comparison	17
5.2	Smart Charging	17
5.2.1	Problem Framing	17
5.2.2	Markov Decision Process Design	18
5.2.3	Agent: Deep Q-Network	19
5.2.4	Training Behaviour	19
5.2.5	Learned Policy	20
5.2.6	Evaluation Against Baselines	20
5.2.7	Example Episode	21
5.2.8	Implications for the Client	21
6	Conclusions and Recommendations	22
6.1	What the Analysis Shows	22

6.2	Recommendations for the Fleet Operator	22
6.3	Private or Public Charging	23
6.4	Limitations	23
6.5	Outlook	23

List of Figures

1	H3 hexagonal grid overlaid on Chicago community areas at resolutions 6, 7, and 8.	7
2	Distribution of trip duration, distance, fare, tips, total cost, and inter-trip idle time across the full dataset.	8
3	Choropleth maps of pickup demand aggregated at community area level and H3 resolutions 6, 7, and 8.	9
4	Pickup demand by 4-hour time bucket at H3 resolution 6, covering the full 24-hour cycle.	10
5	Pickup demand heatmap cross-tabulated by hour of day and day of week.	11
6	Origin–destination matrix for the top 10 community areas by total trip volume. .	11
7	Net flow (pickups-dropoffs) by H3 hexagon, aggregated across the full observation period.	12
8	Training and validation loss curves for the final neural network model.	16
9	Training curves: episode reward, final SoC, cost and penalty over 5,000 training episodes (100-episode moving average).	19
10	Learned charging policy across SoC and time slot: preferred action (off / low / medium / high) per state.	20
11	Distributions over 1,000 evaluation episodes: total reward, final SoC, and cost vs. penalty, compared across DQN Agent, Always-Max, and Random.	21
12	Example evaluation episode: SoC, charging power, and cost per 15-minute slot from 2 p.m. to 4 p.m.	21

1 Introduction

1.1 Business Context

A German automotive manufacturer is launching a ride-hailing platform for US customers, operated entirely by an electrified vehicle fleet. It's entering a mature market with no operational track record, so it needs data-driven guidance on how to manage electric taxi fleets in cities. Bane & Wayne Partners (BWP) was engaged to provide that guidance, and this report presents the work of BWP's Data Science Team, using historical Chicago ride-hailing data to characterize demand patterns, forecast them, and design a charging strategy for electric vehicles.

1.2 Business Goal and Data Mining Goal

The goal is to give the client actionable intelligence for launching and running an electric fleet in a major US city. The analysis answers three questions:

1. **Where and when is demand concentrated?** This drives fleet positioning, driver scheduling, and market entry prioritization.
2. **How accurately can future demand be forecast?** Better predictions cut idle time, lower per-trip energy use, and raise driver earnings.
3. **How should charging be managed to minimize cost?** Charging timed to electricity prices and predicted demand can cut operating cost sharply.

Answering them draws on three data sources, described in detail in the next section: historical Chicago taxi trips from the Chicago Data Portal, hourly weather from Open-Meteo, and OpenStreetMap points of interest for urban context. The work itself is split into four tasks. Task 1 cleans the raw trips, discretizes them onto Uber H3 hexagons, and enriches them with weather data. Task 2 uses descriptive spatial analytics to surface demand hotspots and flow patterns. Task 3 trains two models (a Support Vector Machine and a feedforward neural network) to forecast demand per hexagon and time bucket. Task 4 develops a Reinforcement Learning agent for smart-charging policies under stochastic energy demand and dynamic pricing.

2 Data Description

Three data sources feed the analysis: Chicago taxi trip records, hourly weather, and OpenStreetMap points of interest. Together they cover demand, environmental context, and urban structure.

2.1 Taxi Trip Data

Downloaded from the Chicago Data Portal on 10 May 2026, the dataset holds every taxi trip from January 2024 to early May 2026: **14,985,679 trips** across 21 columns. Seven temporal features (date, hour, day of week, month, year, week, and whether a day is a weekend) are derived at load time, bringing the working total to 28. Table 1 lists the key fields.

Table 1: Core fields of the Chicago taxi trip dataset.

Column	Type	Description
trip_id	string	SHA-1 hash; unique journey identifier

Column	Type	Description
taxi_id	string	SHA-1 hash; anonymized vehicle identifier
trip_start_timestamp	datetime (UTC−6)	Rounded to the nearest 15 minutes
trip_end_timestamp	datetime (UTC−6)	Rounded to the nearest 15 minutes
trip_seconds	float	Recorded journey duration in seconds
trip_miles	float	Recorded journey distance in miles
pickup_census_tract	float	Census tract of pickup ($\approx 54\%$ missing)
dropoff_census_tract	float	Census tract of dropoff ($\approx 56\%$ missing)
pickup_community_area	float	Community area of pickup (0.05% missing)
dropoff_community_area	float	Community area of dropoff (7.2% missing)
fare	float	Metered fare in USD
tips	float	Tip amount in USD
trip_total	float	Total charge billed to the passenger
payment_type	string	Credit card, cash, mobile, etc.
company	string	Taxi company name
pickup_centroid_lat/lon	float	Centroid of the pickup census tract
dropoff_centroid_lat/lon	float	Centroid of the dropoff census tract

Three portal design choices matter downstream. Both IDs are SHA-1 hashes, so there’s no personally identifiable information. All timestamps are rounded to the nearest 15-minute interval for privacy. This caps temporal precision at 15-minute bins but doesn’t affect hourly or 4-hourly aggregations. And location is given only at the census tract level: with roughly 800 tracts in Chicago, the spatial grain is coarser than GPS, but each tract’s centroid lat/lon supports spatial joins.

2.2 Weather Data

Hourly weather comes from the Open-Meteo Historical Archive API (Open-Meteo, [2024](#)) over the same date range as the taxi data (January 2024 - May 2026). Because Chicago spans roughly 600 km² and exhibits meaningful microclimatic variation between the lakefront, the urban core, and the far northwest, a single weather station would not be representative. We therefore defined three representative weather zones by K-means clustering ($k = 3$) on all taxi pickup coordinates, placing zone centroids where demand activity is densest: the downtown core (41.886°N, 87.629°W), the O’Hare area (41.979°N, 87.903°W), and the South Side (41.785°N, 87.750°W).

Table 2: Hourly weather variables fetched from Open-Meteo.

Variable	Unit	Description
temperature_2m	°C	Air temperature at 2 m height
apparent_temperature	°C	Feels-like temperature (wind chill / heat index)
precipitation	mm	Total precipitation
rain	mm	Liquid precipitation
snowfall	cm	Snowfall accumulation
snow_depth	cm	Snow depth on ground
windspeed_10m	km/h	Wind speed at 10 m
windgusts_10m	km/h	Wind gust speed at 10 m
cloud_cover	%	Fractional cloud cover

The dataset contains **61,335 observations** (~20,445 hours for each zone). Temperatures range from -26.8°C to 36.3°C , with apparent temperatures as low as -32.9°C due to wind chill, which is consistent with Chicago’s continental climate. Median hourly precipitation is near zero while median cloud cover stands at 83 %, reflecting the city’s characteristically overcast conditions.

2.3 Point-of-Interest Data

POI data comes from OpenStreetMap via `osmnx` (Boeing, 2017), querying four tag categories for Chicago: `amenity`, `shop`, `tourism`, and `leisure`. The raw tags (e.g. restaurant, bus_stop, hospital) are too fine-grained to use directly, since many are too sparse for stable per-hexagon counts, and their demand signals overlap (e.g. café and bar both mean nightlife). We collapsed them into seven demand-relevant categories, grouping tags with similar expected demand timing and dropping tags too niche to matter. After converting polygons to centroids and discarding unmapped types, **9,124 POIs** remain (Table 3).

Table 3: POI counts by functional category (OpenStreetMap, Chicago).

Category	Count	Examples
Food & Nightlife	5,640	Restaurants, bars, cafés, fast food
Education	1,219	Schools, universities, libraries
Shopping	1,084	Supermarkets, clothing, electronics
Healthcare	469	Hospitals, clinics, pharmacies
Entertainment	348	Theaters, cinemas, museums
Accommodation	229	Hotels, hostels
Transport	135	Bus stops, subway entrances, ferry terminals

POIs are static: they don’t vary over time and aren’t counted as demand. They are aggregated to H3 resolution 8 as per-category counts and joined into the feature matrix as a proxy for how much each cell attracts trips.

3 Data Preparation

3.1 Taxi Data Cleaning

Public portal data needs validation before use. Rather than applying fixed rules upfront, we worked through the taxi dataset step by step, surfacing each quality issue, diagnosing its cause, and choosing a justified fix; in total, cleaning removes just over 10 % of raw records, retaining 89.99 % for analysis. The first issue is missing values, where different fields are missing for different reasons, so each gets its own treatment (Table 4).

Table 4: Missing-value handling by field.

Field	Missing	Cause	Treatment
Timestamps	< 0.01 %	Fall in the autumn DST fold, where one hour repeats and the occurrence is ambiguous	Remove
Drop-off community area	7.2 %	Drop-off outside the 77 official community areas (suburban)	Recover via spatial join where inside the city; retain suburban trips unassigned
Trip duration	small	Not reported but derivable	Impute from end – start timestamp
Trip distance	small	Not reported	Estimate with great-circle distance (understates road distance)
Vehicle ID	8 records	Unattributable to any taxi	Remove

3.1.1 Consistency Checks

Beyond missing values, some records are present but aren’t real journeys. Ghost trips have zero seconds, identical start and end times, and identical start and end locations, which looks like a meter switched on and off; all four conditions must hold, so genuine short trips survive, and we remove the **153,078** records that qualify (~1 %). A further **1,014,515 trips** (6.8 %) log zero distance despite positive duration, with a median of 46 seconds and a quarter under 6 seconds. These are overwhelmingly meter cancellations or GPS failures rather than real journeys, so any trip with zero recorded distance is excluded as well. We also confirmed no end-before-start inversions and no duplicate trip IDs.

3.1.2 Billing Discrepancy

The billed total and the sum of its components (fare, tips, tolls, extras) disagree in 47.9 % of rows. The gap is always positive, almost always exactly \$0.50, and concentrated in credit-card and mobile payment transactions. This pattern is consistent with a card processing surcharge that’s added to the total but is not itemized. We apply no correction and treat `trip_total` as the authoritative revenue figure.

3.1.3 Outlier Removal

After dropping non-journeys, we remove physically implausible values using two derived metrics, average speed and fare per mile, with the thresholds and resulting counts listed in Table 5. The criteria are deliberately conservative, and as a check, the correlation between per-neighborhood pickup counts before and after outlier removal is 1.0000, meaning the spatial demand signal is fully preserved.

Table 5: Outlier removal criteria applied to the taxi trip dataset.

Criterion	Records removed	Share of dataset
Non-positive fare	5,243	0.04 %
Distance exceeding 100 miles	960	0.01 %
Duration exceeding 3 hours	11,629	0.08 %
Average speed above 70 mph	6,663	0.05 %
Average speed below 2 mph	155,478	1.13 %
Fare below \$2 per mile	28,537	0.21 %
Fare above \$100 per mile	small	< 0.01 %

3.1.4 Two Cleaning Levels

Not every task requires the same level of cleaning: for **demand forecasting**, a trip with an odd speed or fare is still a real demand event, whereas for **trip-level analyses** like duration or fare modeling those same records distort results. We therefore keep two versions, and use them accordingly: demand forecasting (Sections 4-5) draws on the minimal level, while the profitability analysis uses the full one.

Table 6: Rows retained under each cleaning level.

Level	What is removed	Rows retained
Demand (minimal)	DST-ambiguous timestamps	14,984,551 (99.99 %)
Trip (full)	Above, plus missing vehicle ID, ghost trips, zero-distance trips, and speed/fare outliers	13,485,728 (89.99 %)

3.2 Weather Data Preparation

The three weather zones from Section 2 come from K-means ($k = 3$) on all pickup locations, with both the elbow method and the silhouette score pointing to $k = 3$. Two further adjustments follow from inspecting the raw hourly series. The data carries a WMO condition code (0–99), but treating it as a number implies a false ordering (code 95 isn’t “95× worse” than code 1), and only three conditions actually appear: clear, rain, snow. Since rain and snow already have their own numeric columns, the code adds nothing and is dropped. Each zone also shows one-hour gaps on spring-forward day (2024-03-10, 2025-03-09), but the 02:00 hour doesn’t exist when clocks jump 01:00 → 03:00, so the gap is correct and no imputation is needed.

3.3 Point-of-Interest Data Preparation

The OSM query returns points, polygons, and lines. All non-point features are reduced to their centroid, so every POI is a single coordinate. Each is mapped to one of seven functional categories via a lookup table, with unmapped types discarded. The final 9,124 POIs are assigned to the hexagon containing their coordinate.

3.4 Feature Engineering

The models don't run on individual trips. Trips are aggregated to a grid: one row per (*hexagon, time bucket*), with pickup count as the target. Cells with no pickups are kept as explicit zero-demand rows, so the model learns from quiet periods too.

3.4.1 Spatial and Temporal Granularity

Grain sets a trade-off: finer units capture local variation but leave more cells at or near zero, which is harder to forecast, and Table 7 shows how sharply that sparsity grows.

Table 7: Zero-demand share across spatial and temporal granularities.

Spatial unit	Time bucket	Grid cells	Zero-demand share
H3 resolution 5 (large hexagons)	1 hour	163,400	5.8 %
H3 resolution 6	1 hour	551,475	26.8 %
H3 resolution 6	4 hours	137,889	12.6 %
H3 resolution 7	4 hours	653,696	49.2 %

We chose **H3 resolution 6 with 4-hour buckets** as our main resolution for now based on the relatively low zero-demand share: 137,889 cells (27 hexagons $\times$ 5,107 time buckets) at a manageable 12.6 % zero-demand share. Resolution 7 pushes zero-demand close to 50 % without a matching gain in operational precision. Throughout the report, we show results at this spatio-temporal resolution, but we also explore the effect of coarser and finer resolutions in Section 4.3.

3.4.2 Features Used in the Forecasting Models

Every feature is knowable before the bucket begins, which rules out anything derived from the demand being predicted, and together the four groups below give 51 input variables per grid cell:

- **Calendar context** - hour, day of week, month, weekend flag, US public holiday flag. These carry the daily and weekly rhythms from the descriptive analysis.
- **Cyclic encoding** - calendar variables are circular (23:00 is next to 00:00, December next to January), so each becomes a sine-cosine pair rather than a plain integer that implies a false break.
- **Weather conditions** - the nine variables for the nearest zone during the bucket, known in advance from forecasts.
- **Spatial identity** - a binary indicator per hexagon, letting the model learn a cell-specific baseline without assuming how neighboring cells relate.

4 Descriptive Analytics

4.1 Temporal and Trip Patterns

Before examining spatial structure, it is useful to characterize how demand is distributed across time, since taxi demand in Chicago is far from uniform across the day or week: it follows predictable rhythms tied to work schedules, social activity, and weather, and those rhythms are a prerequisite for fleet positioning and driver scheduling.

On weekdays, demand follows a clear bimodal pattern: a morning peak around 08:00 driven by commuter traffic and a stronger, broader evening peak between 16:00 and 19:00. Overnight demand (00:00–06:00) is uniformly low across all days of the week, making this window a natural candidate for vehicle charging or maintenance. Weekends exhibit a fundamentally different profile: the morning commuter spike is absent, demand builds gradually through the afternoon and peaks in the early evening, reflecting leisure rather than work-driven travel. Midweek (Tuesday to Thursday) consistently records the highest absolute trip volumes.

4.2 Spatial Analytics

The spatial analysis characterizes where demand originates and how it is distributed across Chicago’s geography. We work from coarse neighborhood-level aggregations down to fine-grained hexagonal cells, so that patterns masked by administrative boundaries become visible, and then complement the static demand maps with time-sliced views, origin-destination flows, and net-flow balances. The last of these carries the most operational weight, since it separates areas that generate trips from those that merely absorb vehicles.

4.2.1 Spatial Discretization: H3 Hexagonal Grid

To enable a consistent and scalable spatial analysis, Chicago’s geography was discretized using Uber’s H3 hierarchical hexagonal grid system at resolutions 6, 7, and 8. Unlike census tracts, which follow irregular administrative boundaries, hexagons offer uniform area coverage and adjacency properties that are well-suited for spatial modeling.

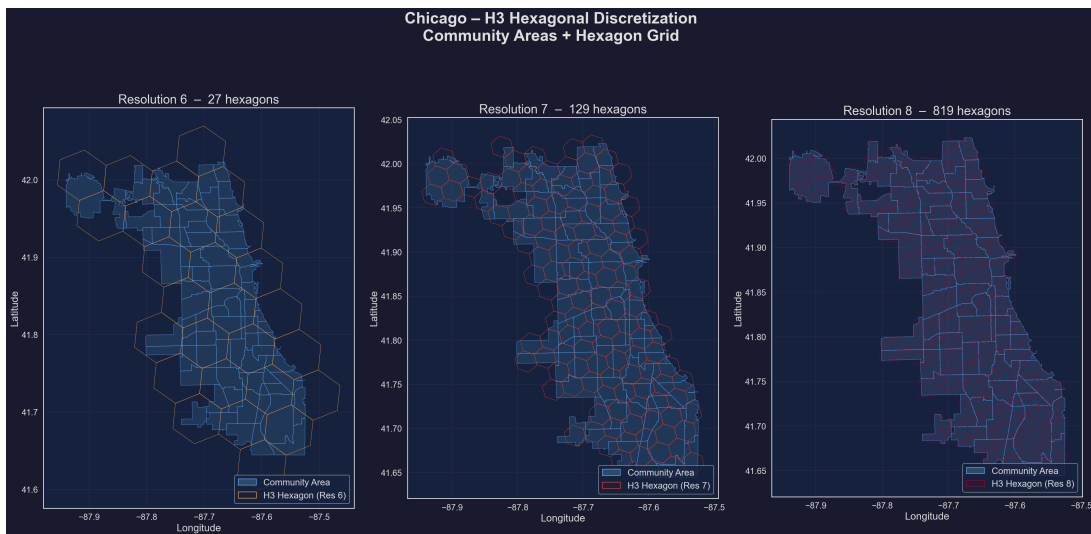


Figure 1: H3 hexagonal grid overlaid on Chicago community areas at resolutions 6, 7, and 8.

Higher resolutions increase the number of hexagons exponentially while reducing their individual area, enabling finer-grained demand localization at the cost of data sparsity per cell. Resolution 7 was chosen as the primary unit for the subsequent analysis as it strikes a balance between spatial granularity and sufficient trip counts per hexagon for stable estimates.

4.2.2 Trip Metric Distributions

Before diving into spatial patterns, we first characterize the overall distribution of key trip metrics. All distributions are clipped at the 99th percentile to prevent outliers, such as airport flat-rate rides or exceptional idle gaps, from distorting the visualization.

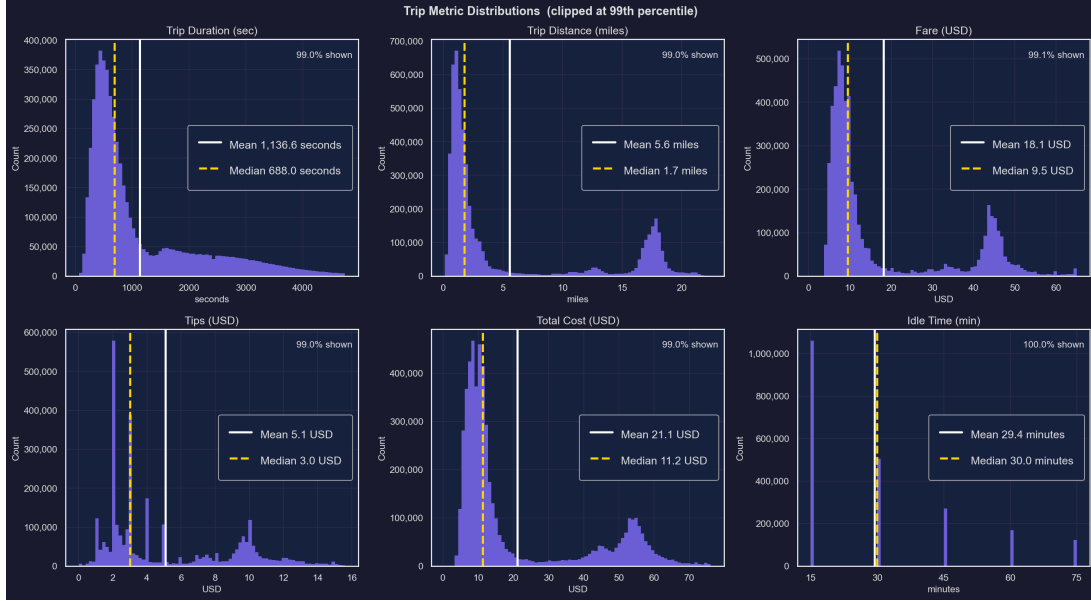


Figure 2: Distribution of trip duration, distance, fare, tips, total cost, and inter-trip idle time across the full dataset.

Several patterns stand out. Trip durations are right-skewed with a median around 600–700 seconds (~10 minutes), indicating that the majority of rides are relatively short urban hops. Trip distances mirror this pattern, with most trips under 5 miles. Fare and total cost distributions confirm the dominance of low-cost short rides, with a long tail driven by airport and cross-city trips. The tip distribution has extraordinarily high counts at specific tip amounts, which could reflect the large share of cashless predefined tips. Idle time between consecutive trips per driver is concentrated in the 0–30 minute range, suggesting reasonably efficient utilization, though the mean around 60 minutes indicates periods of deliberate waiting or repositioning.

4.2.3 Demand by Community Area and Hexagon Resolution

To understand how spatial aggregation affects observed demand patterns, we compare pickup counts across three levels: community area, H3 resolution 6, and H3 resolutions 7 and 8.

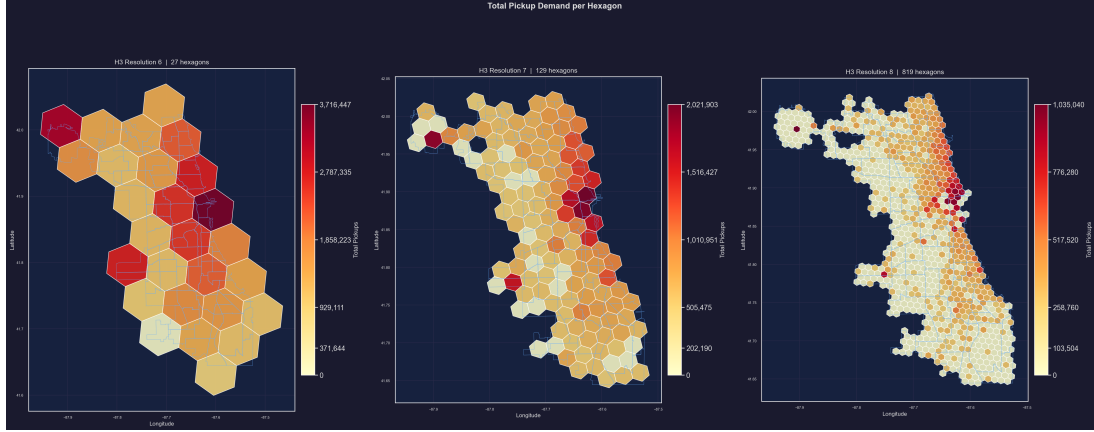


Figure 3: Choropleth maps of pickup demand aggregated at community area level and H3 resolutions 6, 7, and 8.

At the community area level, the Near North Side and the Loop stand out as the dominant demand centers, consistent with their role as Chicago’s central business and entertainment district. The pattern is also visible in the airport community areas, particularly O’Hare in the northwest, which generates a structurally distinct demand cluster far from the city center.

As resolution increases from H3-6 to H3-8, the hotspots sharpen considerably. At resolution 7, the spatial signal localizes to specific corridors: the Magnificent Mile, the River North entertainment district, and the area immediately surrounding O’Hare. Zones with moderate activity at coarser resolutions, such as the South Loop or Wicker Park, reveal internal heterogeneity at finer scales, with some sub-areas generating substantially higher demand than their neighbors. This demonstrates that aggregation at the community area level masks actionable sub-neighborhood variation that is relevant for fleet positioning decisions.

4.2.4 Temporal Demand Patterns Across Space

Demand does not only vary spatially but also shifts throughout the day. To capture this, pickups were aggregated into 4-hour time buckets at H3 resolution 6, and a choropleth map was produced for each bucket.

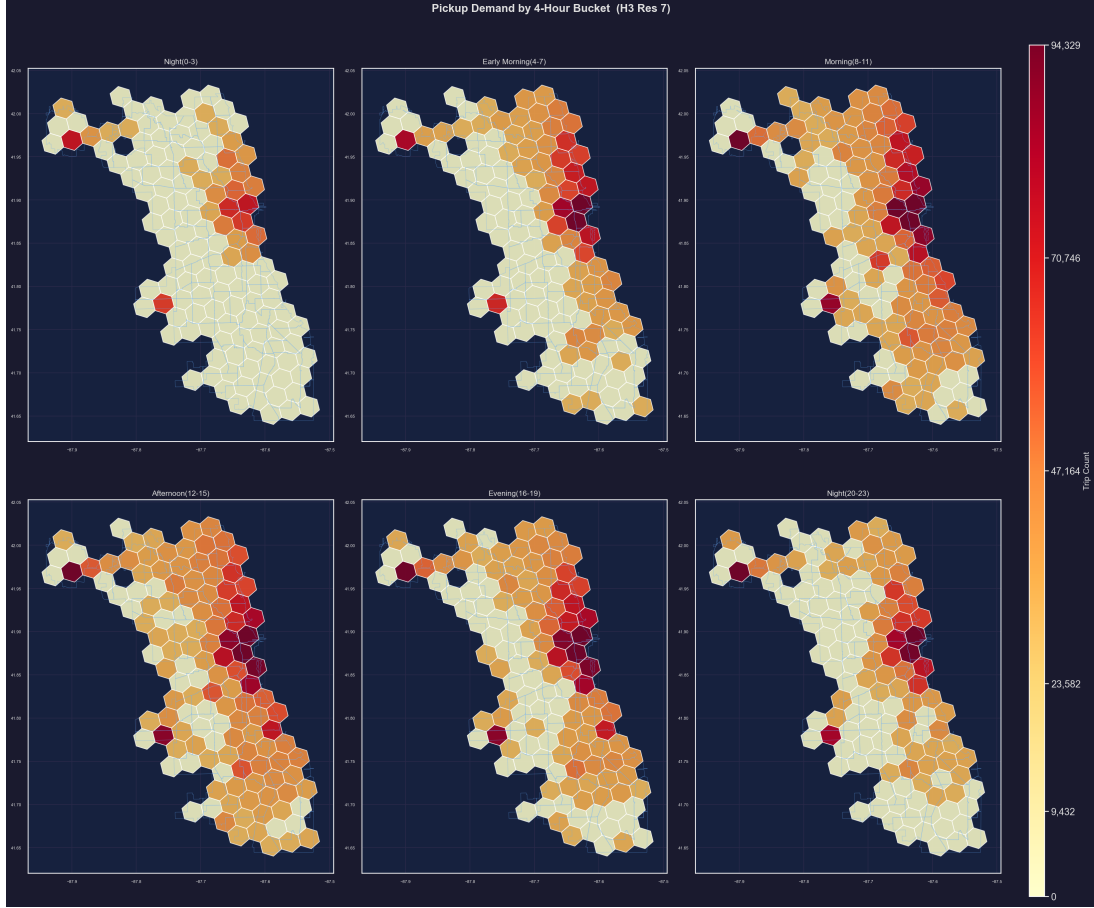


Figure 4: Pickup demand by 4-hour time bucket at H3 resolution 6, covering the full 24-hour cycle.

The northeastern lakefront and downtown corridor is the dominant demand hub across all time buckets, confirming its role as a structurally persistent hotspot regardless of the time of day. During the night window (00:00–03:59), overall demand is low but the downtown core retains relative activity, consistent with late-night hospitality and entertainment traffic. The early morning bucket (04:00–07:59) reveals a distinct secondary hotspot around Midway Airport in the southwest, attributable to early departures. The morning and midday buckets (08:00–15:59) see demand spread more broadly across the eastern half of the city, reflecting a mix of commuter, business, and leisure travel. The afternoon and evening windows (12:00–19:59) represent the peak demand period, with the downtown core intensifying sharply while outer neighborhoods remain comparatively quiet. From 20:00 onward, activity tapers off but the lakefront strip retains a persistent hotspot driven by restaurant and nightlife traffic.

4.2.5 Demand Across the Weekly Cycle

The 4-hour maps show how demand moves over a single day, but the weekly structure is easier to read in a single cross-tabulation. The heatmap below tabulates total pickups by hour of day against day of week, compressing the whole observation period into one view.

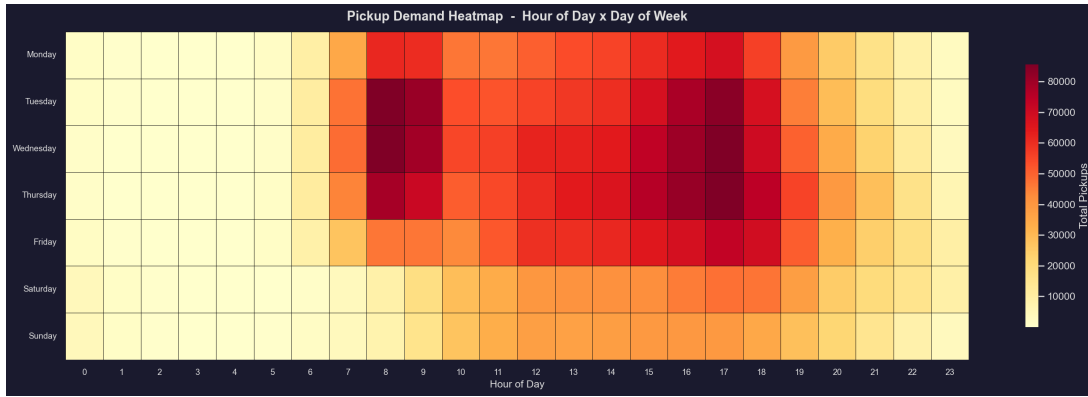


Figure 5: Pickup demand heatmap cross-tabulated by hour of day and day of week.

The bulk of trips occur between 07:00 and 19:00 on weekdays, forming a dense demand block from Tuesday to Thursday that represents the highest observed demand in the dataset. Weekends show a distinctly different rhythm: demand starts later in the morning, lacks a pronounced commuter spike, and trails off more gradually in the evening, consistent with leisure rather than work-driven travel. The 00:00–06:00 window is uniformly quiet across all days, making it a natural candidate for driver rest periods or vehicle charging windows in an electric fleet context.

4.2.6 Origin–Destination Analysis

To understand mobility flows between areas, an origin–destination (OD) matrix was constructed for the top 10 community areas by outgoing trip volume.

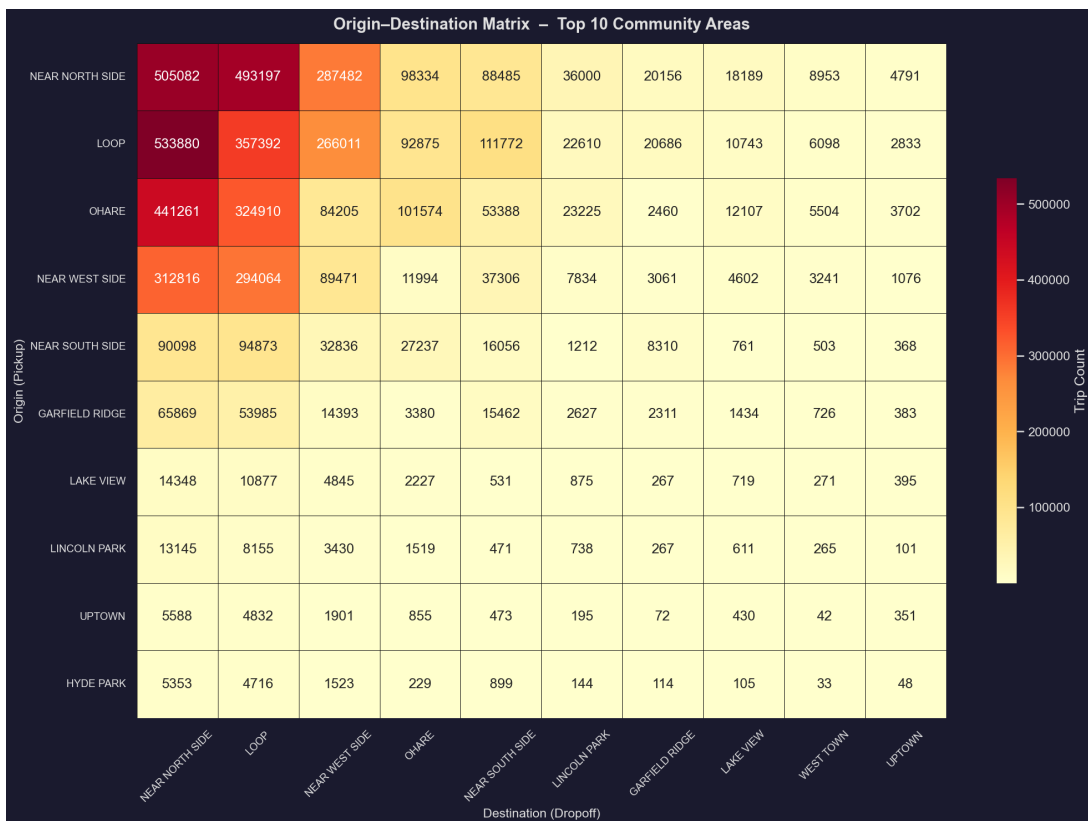


Figure 6: Origin–destination matrix for the top 10 community areas by total trip volume.

The Near North Side and the Loop dominate both as origins and as destinations by a wide margin, reinforcing their central role in Chicago’s taxi network. Despite its geographic isolation in the northwest, O’Hare International Airport generates nearly 500,000 trips directed toward the Near North Side and Loop alone, confirming the airport-to-downtown corridor as one of the most important in the network. For a fleet operator, this could imply that vehicles dropping off passengers in outer neighborhoods face a structural repositioning challenge to return to high-demand areas, in comparison to trips on the main demand corridor of the city.

4.2.7 Spatio-Temporal Net Flow Analysis

Beyond raw pickup and dropoff counts, net flow, defined as pickups minus dropoffs per hexagon, reveals where demand is structurally generated versus absorbed. This metric is directly actionable for fleet operators: zones with persistently positive net flow indicate areas where more vehicles are needed than are being returned by dropoffs, while negative net flow areas signal vehicle accumulation that may require active redistribution.

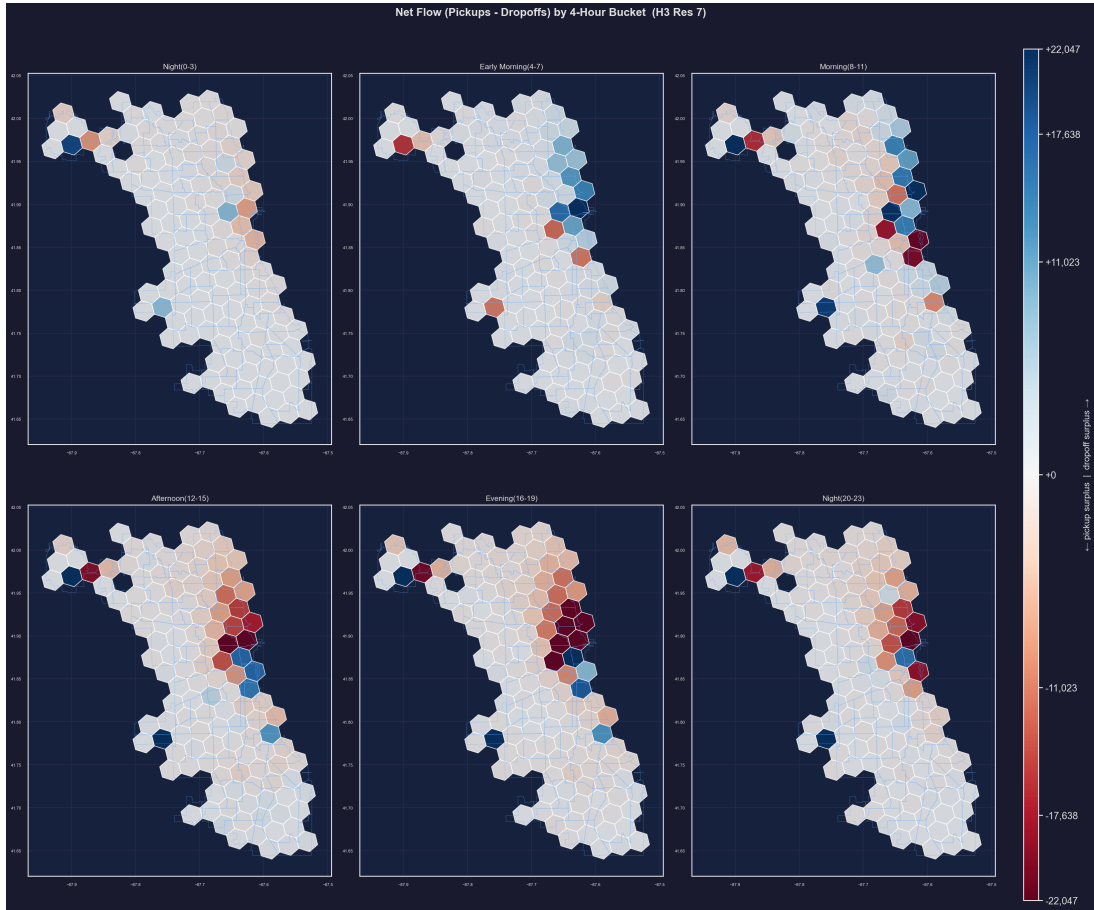


Figure 7: Net flow (pickups-dropoffs) by H3 hexagon, aggregated across the full observation period.

The downtown core and the O’Hare corridor show the most pronounced positive net flow, meaning that taxi demand in these areas consistently exceeds the volume of vehicles returned by dropoffs. The residential outskirts, by contrast, exhibit negative net flow, acting as vehicle sinks that absorb supply without generating equivalent demand in return. For an electric fleet operator, this spatial imbalance has direct implications for charging infrastructure placement: charging points situated in negative-flow residential areas could serve as productive holding

locations, where idle vehicles recharge between shifts, since we expect idle times to be longer there, before being repositioned to high-demand corridors.

5 Data Analytics

5.1 Predictive Analytics

5.1.1 Problem Formulation and Validation Strategy

The forecasting task is to predict **taxi demand per spatial unit and time window**, which we define as the number of taxi trips that start in a given hexagon during a given time period, using only information that is available before the window begins: time of day, day of week, weather forecast, and location. We want to answer questions like “what demand can we expect on a rainy Sunday from 12:00–16:00 in the outskirts?”, not to project forward from the previous window’s actual counts, which a planner would not have.

Two properties of the target shape the setup that follows. Trip counts are zero-inflated and right-skewed, with a few downtown cells recording hundreds of pickups per window while most outer cells record single digits, so we log-transform the target before training to stabilize the variance and report all metrics on the back-transformed count scale to keep them interpretable in trips. And because no model uses lagged demand, no temporal signal can leak through row order, which lets us shuffle the demand grid and split it 50 / 20 / 30 instead of cutting it chronologically. Validation selects model configurations, while the test set is touched only once, for the final metrics.

Two naive benchmarks provide the context against which everything below is read:

- *Per-cell mean*: the historical average demand for each hexagon and time window. MAE = 67.18 trips.
- *Climatology*: the historical average for each hexagon, hour of day, and weekday-vs-weekend combination. MAE = 25.63 trips.

Any model that fails to beat the per-cell mean offers nothing, but climatology is the real hurdle, since it already captures the weekly cycle that drives most foreseeable demand variation.

5.1.2 Support Vector Machine

Kernel Progression and Hyperparameter Tuning

A Support Vector Machine for regression (SVR) fits a function that stays within a tolerance band around the observed values while remaining as smooth as possible. The kernel sets the shape of that function: linear allows only straight-line relationships, polynomial curved ones, and the Radial Basis Function (RBF) highly flexible non-linear relationships. We start simple and add complexity, so that any gain can be attributed to the added flexibility rather than assumed. Alongside the kernel we tune three settings: C , a regularization parameter; epsilon, the width of the no-penalty tolerance band; and gamma, how far each observation’s influence reaches (RBF and polynomial only). Table 8 shows the best result achieved per kernel family on the validation set.

Table 8: Best SVR configuration per kernel type, scored on the validation set.

Kernel type	Best regularization	Validation MAE
Linear	$C = 1.0$	39.58 trips
Polynomial	$C = 10.0$	29.59 trips
RBF	$C = 10.0$	25.51 trips

Moving from linear (MAE 39.58) to RBF (MAE 25.51) shows the value of flexibility: the link between weather, calendar and demand is non-linear, and the RBF kernel captures that curvature. We take the RBF kernel with $C = 10$ and $\epsilon = 0.1$ into final evaluation.

Final Test-Set Evaluation

With the configuration fixed, the model is refit and scored once on the held-out test set of 41,367 grid cells, which is the only point at which that data is used. Table 9 puts the result next to the two baselines.

Table 9: SVR test-set performance compared to baselines.

Model	MAE	RMSE	R^2
Per-cell mean	67.18	216.97	0.633
Climatology	25.63	93.19	0.932
SVR (RBF)	24.33	88.05	0.940

The SVR beats both baselines, but the margin over climatology is rather small. With no access to recent actual counts, most of what the SVR can learn is the same seasonal pattern climatology already encodes, and it has to recover that pattern from general-purpose features instead of reading it off a precomputed schedule. That headline figure also conceals important structure in where the model struggles, so Table 10 breaks the same test set down by demand level.

Table 10: SVR test-set error decomposed by demand level.

Demand level (trips per window)	MAE	Observations
0–1	0.68	7,580
1–5	1.70	7,035
5–20	3.81	10,328
20–100	11.91	8,725
100+	109.89	7,699

Accuracy is high on low-demand cells, which make up the largest share of the test set (14,615 of 41,367 cells sit below 5 trips per window), but the model systematically under-predicts the busy downtown and airport hexagons where forecasts matter most. This is a known artifact of log-transformed regression: training compresses extreme values, so predictions fall short once returned to the original scale.

Effect of Spatial and Temporal Resolution

To assess performance across different granularity choices, we test the tuned configuration across the full grid of three spatial resolutions and three time-window widths. Raw MAE is not comparable across these settings: finer hexagons and narrower windows mechanically hold fewer

trips per cell, so the absolute error shrinks even when skill does not improve. We therefore judge the comparison on R^2 and MAPE, both scale-independent, and report MAE only for reference.

Table 11: SVR performance across spatial and temporal resolutions.

Hexagon size	Time window	MAE	R^2	MAPE
Large (resolution 5, 8 cells)	1 hour	24.57	0.860	0.454
Large (resolution 5, 8 cells)	4 hours	73.37	0.938	0.315
Large (resolution 5, 8 cells)	8 hours	116.64	0.944	0.227
Medium (resolution 6, 27 cells)	1 hour	8.18	0.889	0.582
Medium (resolution 6, 27 cells)	4 hours	31.10	0.902	0.454
Medium (resolution 6, 27 cells)	8 hours	45.73	0.948	0.323
Small (resolution 7, 128 cells)	1 hour	2.98	0.790	0.704
Small (resolution 7, 128 cells)	4 hours	9.50	0.842	0.599
Small (resolution 7, 128 cells)	8 hours	12.69	0.886	0.455

The two dimensions pull in opposite directions. Wider time windows help consistently: at every resolution, moving from 1-hour to 8-hour windows raises R^2 and lowers MAPE, because aggregating over more hours averages out the short-term noise the model cannot see. Finer hexagons hurt: MAPE rises monotonically from resolution 5 to 7 at every window width, and R^2 holds roughly steady between resolutions 5 and 6 before dropping clearly at resolution 7 (0.790 to 0.886, against 0.860 to 0.944 at resolution 5). Splitting the city into 128 cells spreads the same trips more thinly, and with no lag features the model has little left to distinguish neighboring cells. Resolution 6 with 4-hour windows is the balance we keep: it retains enough spatial detail to be operationally useful while staying in the range where the model still performs. Census tracts would mean roughly 800 spatial units, well beyond resolution 7, and the trend here suggests the added sparsity would make things worse still.

Model Shortfalls

Three structural limits explain why the margin over climatology stays thin, and each is a lever for follow-up work. The model has no access to recent actual demand counts, normally the strongest single predictor in demand forecasting; we deliberately exclude lag features to keep the framing purely cross-sectional, but adding them would likely widen the margin considerably. It is also computationally constrained: tuning ran on 8,000 rows and the final refit on 30,000, so the model never saw most of the training data. Finally, the one-hot hexagon encoding treats every cell as independent. Encoding distance to the city center or to the nearest transport hub would let the model learn shared spatial structure instead of memorizing each hexagon in isolation.

5.1.3 Feedforward Neural Network

Architecture and Hyperparameter Search

We train a fully connected feedforward neural network (FFNN) on the same demand grid and feature set as the SVR, so the two are directly comparable. The network takes the 51-variable feature vector as input, passes it through stacked layers of neurons, and returns a single demand estimate. A grid search covers hidden layer sizes ([64, 64], [128, 64], [256, 256] and [64, 64, 64]), dropout (0 %, 10 %), learning rate (0.1, 0.01, 0.001) and optimizer (SGD, Adam). Each candidate trains for up to 200 iterations, stopping early after 20 iterations without validation improvement. The winning configuration is then retrained from scratch on the combined training and validation data.

Activation Function Comparison

Holding the best architecture fixed, we compare three activation functions, the choice that determines how each layer transforms its input.

Table 12: Validation error by activation function for the best network architecture.

Activation function	Validation MAE
ReLU	21.93 trips
Tanh	20.04 trips
Sigmoid	93.29 trips

Tanh edges out ReLU on validation error, and both are far ahead of sigmoid, which never trains to a usable error level. Sigmoid suffers from gradient saturation: its neurons produce near-constant outputs across much of the input range, which stalls learning in deeper networks. The final model uses ReLU. The gap to tanh is under two trips of MAE, small enough to fall within run-to-run variation on a single validation split, so we did not treat it as grounds for switching.

Final Test-Set Evaluation

The best configuration (two hidden layers of 256 neurons each, ReLU activation, no dropout, learning rate 0.001, Adam optimizer) is retrained on all training and validation data and evaluated once on the held-out test set.

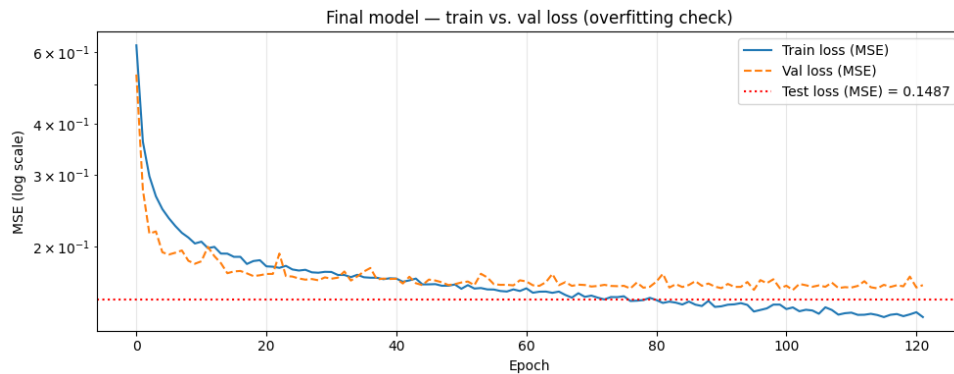


Figure 8: Training and validation loss curves for the final neural network model.

Table 13: Neural network test-set performance compared to the climatology baseline.

Model	MAE	RMSE	R ²
Climatology	24.42	87.22	0.937
Neural network	21.16	80.78	0.946

Like the SVR, the neural network beats climatology, but by a wider margin: MAE 21.16 against 24.42, RMSE 80.78 against 87.22, and R² of 0.946 against 0.937. It therefore captures more of the demand structure that lies beyond the repeating weekly cycle, in particular the non-linear interactions between weather, time of day and location that a fixed seasonal average cannot express. A residual bias of -8.25 trips shows the extreme peaks in the busiest downtown

cells are still missed. Those spikes come from events outside the feature set, such as sports fixtures and concerts, and would need event-calendar data to forecast reliably.

Illustrative Demand Forecasting

Queried for an outer-city hexagon on a rainy Sunday in July, the model predicts demand rising from 34.4 trips at 10:00 to a peak of 37.3 at 12:00, then falling to 20.4 by 18:00, for a total of 281.7 trips across the nine hourly slots. The gradual build-up and decline matches the leisure weekend pattern identified in the descriptive analysis. On-demand forecasts like this, for any combination of location, time, weather, and day type, are the operational output the model delivers to the fleet operator.

5.1.4 Model Comparison

Both models are trained on the same H3 resolution 6 grid with 4-hour windows, so their metrics are directly comparable. Each is also measured against its own climatology baseline; the two notebooks use slightly different random splits, which is why the baseline values differ a little:

Table 14: Each model’s performance relative to the climatology baseline under its own granularity setting.

Model	Setting	Climatology MAE	Model MAE	Improvement
SVR (RBF)	H3 res. 6, 4 h windows	25.63	24.33	Better (+5 %)
Neural network	H3 res. 6, 4 h windows	24.42	21.16	Better (+13 %)

The neural network wins, for two reasons. It trains on the full dataset with mini-batch gradient descent, so it scales with data size, while the SVR works from a small subsample and leaves most of the training information unused. It also learns feature interactions jointly across its layers, whereas the SVR’s fixed kernel applies the same transformation to every input and adapts less well to combinations such as rain plus late evening in airport hexagons.

The added complexity is worth it here. Both models beat the seasonal schedule, but the SVR only marginally, while the neural network improves on it by roughly two and a half times as much. For a fleet operator positioning vehicles under changing weather or on atypical weekdays that difference is the whole point: a seasonal average says nothing about whether tomorrow’s rainy Monday will run busier than a dry one.

5.2 Smart Charging

5.2.1 Problem Framing

Beyond forecasting where demand arises, an electrified fleet must be operated: every vehicle that returns home has to recharge before its next shift, and the way charging is scheduled directly affects operating cost. We model a single electric taxi that arrives home at 2 p.m. and must leave the garage at 4 p.m. Instead of a flat charging rate, the client requested an intelligent agent that sets the charging power every 15 minutes, trading off two competing objectives: keep the recharging cost low, and guarantee enough energy for the upcoming shift, whose demand is unknown at the time charging decisions are made.

This is a sequential decision problem under uncertainty, which is why we frame it as a reinforcement learning task rather than a one-shot optimisation: the agent observes the battery’s state of charge (SoC) at each 15-minute step and must decide the charging power for that step alone, without knowing in advance how much energy the next shift will actually require.

5.2.2 Markov Decision Process Design

Casting the charging problem as an MDP requires committing to a state representation, an action set, and a reward signal. Each choice has consequences for what the agent can learn and how directly the result can be explained to a non-technical client, so we document them here individually.

State space. The state is a 2-dimensional vector: the current SoC in kWh, and the current 15-minute slot index, normalised to $[0, 1]$. Two components are sufficient because the charging window is fixed (2 hours, 8 slots) and the only quantity that evolves within it is the battery level; adding calendar or weather features here would be redundant, since none of them change within a single 2-hour window. Keeping the state low-dimensional also keeps the learned policy interpretable as a simple heatmap over SoC and time slot (Figure 10), which matters for explaining the agent’s behaviour to the client.

Action space. We discretise charging power into four levels — off, low, medium, high (0, 7.3, 14.7, 22 kW) — rather than allowing continuous power output. The assignment explicitly permits this simplification, and discrete actions keep DQN applicable without the added complexity of a continuous-control algorithm (e.g. DDPG), which would need substantially more tuning for a 4-way decision that a home charging unit would realistically only support in discrete steps in practice.

Reward function. Every slot incurs a charging cost that grows exponentially in power, $c \cdot e^{p_t/p_{\max}}$, reflecting that fast charging draws a disproportionate premium on real electricity tariffs (modelled here on average ComEd afternoon supply prices) and stresses the battery more than slow charging. At the end of the window, a shortfall between the realised stochastic demand ($\mathcal{N}(30, 5^2)$ kWh) and the final SoC is penalised at a coefficient of 100 per kWh missing — deliberately far above the cost scale of charging itself, so that the agent never learns to gamble on running out of energy to save a few cents. Table 15 summarises the resulting reward per action, evaluated at the arbitrary reference cost coefficient of the simulation.

Table 15: Charging cost by action, before any shortfall penalty.

Action	Label	Power (kW)	Energy per slot (kWh)	Cost this slot
0	Off	0.0	0.00	4.00
1	Low	7.3	1.83	5.58
2	Medium	14.7	3.67	7.80
3	High	22.0	5.50	10.87

A note on scaling. An earlier version of the cost function used $c \cdot e^{p_t}$ directly. Because p_t ranges up to 22, this exploded numerically and made the reward signal uninformative for the agent. Normalising the exponent by $p_{\max}$, as shown above, keeps the cost in a well-behaved range while preserving the intended convexity — charging faster remains disproportionately expensive, just not to the point of destabilising training.

5.2.3 Agent: Deep Q-Network

We use a Deep Q-Network (DQN) rather than tabular Q-learning or policy-gradient methods, for three reasons directly tied to this problem’s structure. First, the action space is small and discrete (four choices), which is exactly the regime Q-learning-based methods are strongest in; policy-gradient methods (REINFORCE, Actor-Critic) add complexity that buys nothing here. Second, the state space is continuous (SoC is a real-valued kWh amount), so tabular Q-learning would require discretising it and lose precision at the SoC boundaries near the shortfall threshold. Third, DQN’s experience replay lets the agent learn from individual 15-minute transitions rather than waiting for full episodes to complete, which is more sample-efficient than Monte Carlo approaches given the small number of decision points (8) per episode.

The Q-network is a small feedforward network with two hidden layers (12,932 parameters total), trained with a target network and soft updates ($\tau = 0.0005$) for stability, and an ε -greedy exploration schedule decaying from 1.0 to 0.001 over training. Training ran for 5,000 episodes (40,000 transitions).

5.2.4 Training Behaviour

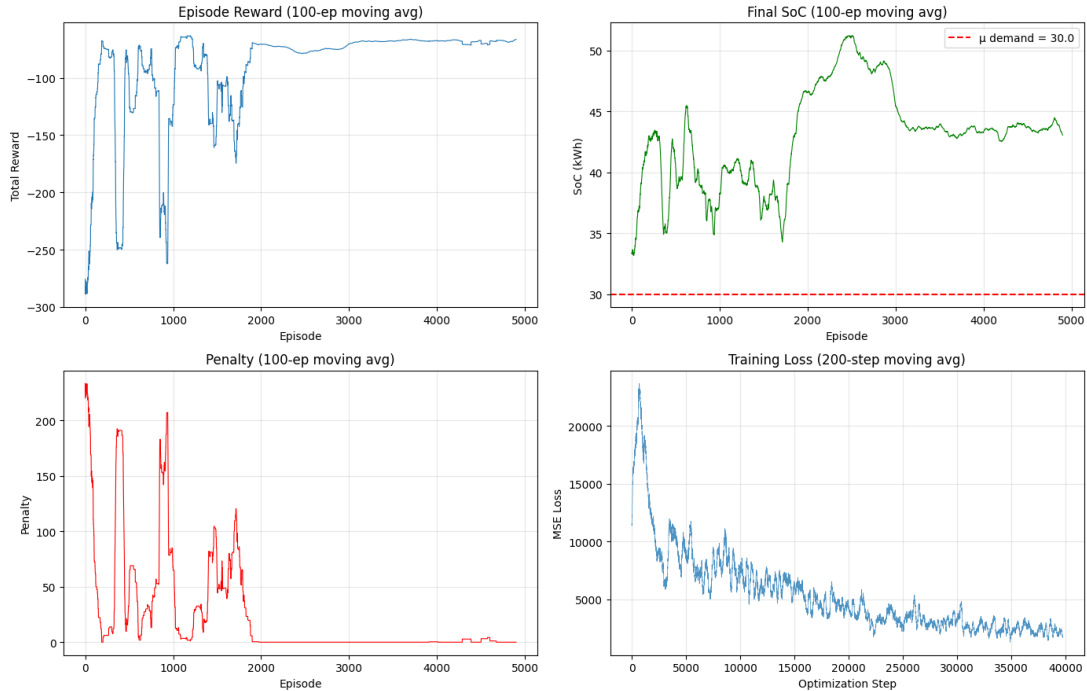


Figure 9: Training curves: episode reward, final SoC, cost and penalty over 5,000 training episodes (100-episode moving average).

Figure 9 shows the training curves. Reward and shortfall penalty do not decline monotonically: several stretches (around episodes 500 and 1,000) show a spike in average penalty, where the agent temporarily under-charges before correcting. This reflects the exploration-driven trade-off inherent to ε -greedy DQN — the agent must occasionally under-charge to learn the shape of the penalty, since the shortfall penalty is only observed at the end of an episode. By the later episodes, the average penalty converges to zero while average final SoC stabilises above the 30 kWh demand mean, indicating the agent has learned a conservative buffer against demand uncertainty rather than charging to the mean exactly.

5.2.5 Learned Policy

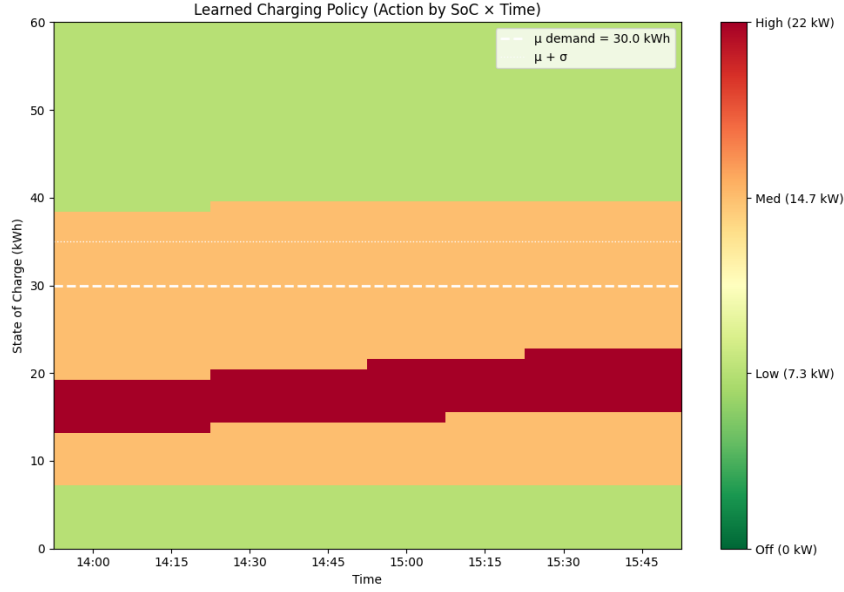


Figure 10: Learned charging policy across SoC and time slot: preferred action (off / low / medium / high) per state.

Figure 10 visualises the agent’s preferred action across the full state space (SoC on one axis, time slot on the other). The policy is intuitive from an operations standpoint: at low SoC the agent charges at high power regardless of the time slot remaining, since running out of runway to reach a safe SoC is the single most expensive outcome available to it (the shortfall penalty dominates any charging cost). As SoC rises, the agent shifts to lower power levels and eventually switches off entirely once SoC comfortably exceeds the expected demand. Because the reward is asymmetric — a large one-sided penalty for shortfall versus a comparatively small, smoothly increasing cost for charging — the policy is risk-averse by construction: it consistently charges to a small surplus above the 30 kWh mean rather than charging exactly to the expected demand, which would leave a roughly 50% chance of a shortfall.

5.2.6 Evaluation Against Baselines

We evaluate the trained agent against two baselines over 1,000 episodes each, to average out the stochastic demand draw: **Always-Max**, which charges at 22 kW every slot regardless of need, and **Random**, which selects actions uniformly. Table 16 reports the results.

Table 16: Policy comparison over 1,000 evaluation episodes.

Policy	Avg. Reward	Avg. Cost	Avg. Penalty	Avg. Final SoC (kWh)	Shortfall rate
DQN Agent	-65.10	64.45	0.65	42.8	0.4 %
Always-Max	-86.99	86.99	0.00	56.5	0.0 %
Random	-202.91	56.39	146.52	34.4	29.1 %

The DQN agent dominates on total reward, the metric that matters operationally because it combines both objectives. Always-Max never runs short, but pays for a permanent safety margin it does not need — it ends slots with an average SoC of 56.5 kWh against a 30 kWh mean

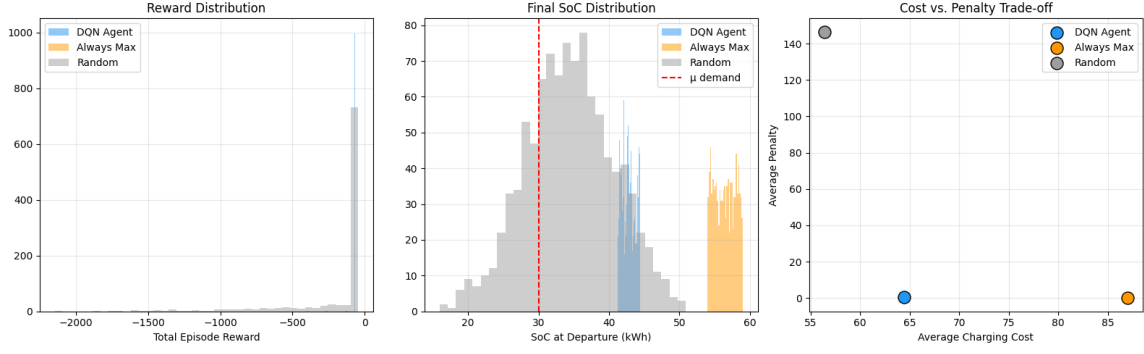


Figure 11: Distributions over 1,000 evaluation episodes: total reward, final SoC, and cost vs. penalty, compared across DQN Agent, Always-Max, and Random.

demand, paying roughly 35% more in charging cost than the DQN agent for a shortfall guarantee the DQN agent already delivers at 99.6% reliability. Random illustrates the downside of no policy at all: a 29.1% shortfall rate would mean roughly one in three shifts starting under-charged, which is not an acceptable outcome for a ride-hailing operator’s service levels. The DQN agent sits close to Always-Max’s safety with a cost profile much closer to Random’s — it has learned to charge no more than the risk of shortfall requires.

5.2.7 Example Episode

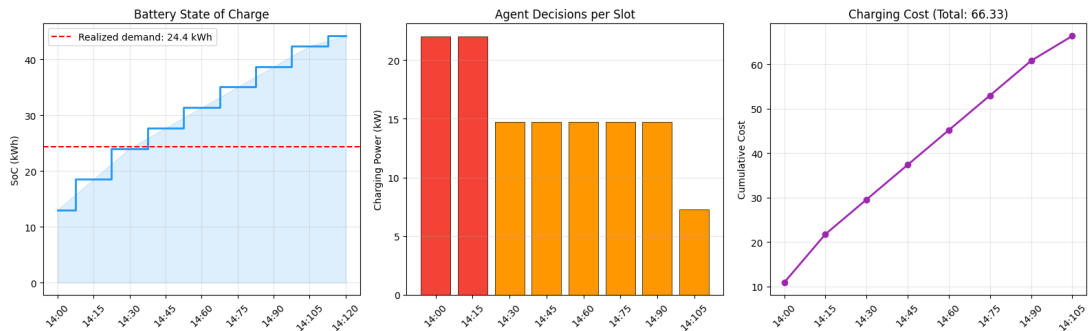


Figure 12: Example evaluation episode: SoC, charging power, and cost per 15-minute slot from 2 p.m. to 4 p.m.

Figure 12 traces one evaluation episode slot-by-slot: SoC rises from an initial value in the 10–15 kWh range at 2 p.m. to a final SoC of 44.2 kWh at 4 p.m., against a realised demand draw of 24.4 kWh — a comfortable 19.8 kWh surplus at a total charging cost of 66.33. This single trace should not be read as typical (the realised demand here happened to fall below the mean), but it illustrates the slot-by-slot decision pattern: higher power early in the window while SoC is low, tapering off as the buffer builds.

5.2.8 Implications for the Client

For the fleet operator, the practical takeaway is that a static “charge to full” policy is not cost-optimal even though it is the safest option available: it is more expensive to run at scale than a policy that models demand uncertainty explicitly, without a meaningfully lower risk of an under-charged vehicle leaving the garage. The learned policy’s shape — charge aggressively only when SoC is genuinely low, taper as a buffer builds — could be implemented directly as a

lookup table on the vehicle’s home charging unit, since the state space is only two-dimensional and the policy is stable across it.

Two caveats are worth stating plainly to the client. First, the agent was trained on a single, fixed demand distribution ($\mathcal{N}(30, 5)$); if realised trip demand systematically shifts (e.g. due to a new city contract or a seasonal change in ride volume), the policy would need retraining on updated demand data, since it has not learned to infer demand from context, only to hedge against a fixed distribution. Second, the penalty coefficient of 100 kWh^{-1} was chosen to strongly discourage shortfalls; a lower coefficient would shift the policy toward cheaper, riskier charging, and the client should treat this coefficient as a dial to be set according to their actual tolerance for an under-charged vehicle at shift start, not as a fixed technical parameter. A brief sensitivity sweep over this coefficient would be a natural next step before deployment, and is available on request.

6 Conclusions and Recommendations

6.1 What the Analysis Shows

Taken together, the analyses paint a consistent picture of how ride-hailing demand behaves in Chicago and what an incoming electric fleet should do about it. Demand is concentrated in space and structured in time. A small number of hexagons in the downtown core, the Magnificent Mile, and the O’Hare corridor account for a disproportionate share of all pickups, while most of the city sits close to zero for much of the day. On top of that spatial structure sits a predictable weekly rhythm: a bimodal weekday pattern with a morning commuter peak and a stronger evening peak, a later and flatter weekend profile, and a quiet overnight window from midnight to six that is common to every day of the week.

These patterns are the foundation for the two forecasting models and the charging agent. Both forecasting models beat the seasonal climatology baseline: the feedforward neural network by a clear margin (MAE 21.16 against 24.42, R^2 0.946 against 0.937) and the Support Vector Machine only marginally. The reinforcement learning agent learned a charging policy that reliably met the driver’s departure energy needs at a cost well below a naive charge-at-maximum strategy. The sections below translate these results into recommendations for the client and set out where the analysis stops short.

6.2 Recommendations for the Fleet Operator

Position vehicles for the corridors, not the map as a whole. The strongest and most repeatable finding is that demand lives in a handful of places. For a fleet entering the market, this argues for a phased rollout that starts in the downtown-to-lakefront corridor and the airport connections rather than an even spread across all 77 community areas. The origin-destination analysis reinforces this: O’Hare alone sends close to half a million trips a year toward the Near North Side and the Loop, so the airport-to-downtown axis is a dependable source of paid mileage from day one.

Use the net-flow signal to plan repositioning. Pickups and dropoffs do not balance locally. The downtown core and the airport run a persistent positive net flow, meaning they hand out more trips than they take back, while residential neighborhoods on the edges absorb vehicles without generating return demand. A fleet that ignores this will accumulate idle cars in low-demand areas during the day. Building repositioning logic around the net-flow map, and treating the outer sinks as places to hold and charge rather than places to wait for fares, addresses this directly.

Forecast for planning, and know what the forecast cannot see. The neural network is worth the added complexity because it responds to weather and to atypical days in a way that a fixed weekly schedule cannot. It is the right tool for shift planning and for answering concrete questions such as expected demand in a given area on a rainy Sunday afternoon. What it does not capture are one-off spikes from concerts, sports fixtures, and conventions, since nothing in the feature set flags those events. The operator should treat model output as a strong baseline to be overridden by hand on known event days, not as a complete demand picture.

6.3 Private or Public Charging

The charging results and the demand geography point to a mixed answer rather than a clean one. The cost function penalizes high charging power convexly, so charging slowly over a long window is much cheaper than charging in short bursts at full rate. The agent exploited exactly this, spreading energy across the available slots and keeping a modest buffer above expected demand instead of racing to a full battery. That economics favors private or depot charging for the bulk of a vehicle’s energy needs: it lines up with the quiet overnight window the descriptive analysis identified, it lets each car draw at a low, cheap rate for hours, and it can sit in the negative-net-flow residential zones where cars would otherwise be idle anyway.

Public fast charging works against this cost structure, because its whole value is delivering a lot of energy quickly, which is the expensive corner of the curve. Our recommendation is therefore to anchor the fleet on private or depot charging placed near where vehicles naturally end their shifts, and to use public fast charging only as an opportunistic top-up during operations, for a car that would otherwise have to leave a high-demand corridor to recharge. Committing entirely to public infrastructure would expose the operator to both higher energy costs and competition for chargers during peak hours.

6.4 Limitations

Several caveats bound how far these findings should be pushed. Location in the source data is reported at census-tract centroids rather than true GPS coordinates, so every hexagon assignment, the hotspot detection, and the flow maps inherit a coarse spatial granularity; fine within-tract patterns are invisible by construction. The forecasting models see no recent demand counts and no event calendar, which is the main reason both under-predict the largest downtown spikes and why the SVR could barely improve on a seasonal average. The SVR faced an additional practical ceiling: it had to be tuned on a small subsample because it does not scale to the full grid, so it was never shown all the available data. Finally, the charging agent is trained against a stylized environment with a fixed cost coefficient and a normally distributed departure demand. Real electricity prices vary hour to hour and real energy needs depend on the next day’s trips, so the learned policy is a proof of concept rather than a deployable controller.

6.5 Outlook

The most valuable next step is to add lagged demand to the forecasting models. In demand forecasting the previous period’s actual count is usually the single strongest predictor, and its absence here is the clearest gap between our setup and a production system. A spatially aware encoding of the hexagons, one that captures distance to the center, proximity to transport hubs, and neighboring demand, would also let the models generalize across cells instead of learning each one in isolation. On the data side, an event calendar covering concerts, conventions, and sporting fixtures would target precisely the spikes the current models miss, and real-time or day-ahead electricity prices would let the charging agent optimize against the tariffs a fleet

actually faces rather than a flat coefficient. Further analyses worth pursuing on the existing data include a profitability study that pairs the demand maps with fare-per-mile and idle-time patterns to find where drivers earn most efficiently, and an explicit repositioning model that turns the net-flow findings into concrete vehicle movements. Each of these builds directly on the groundwork laid here and would move the analysis closer to an operational decision-support system for the client.

References

- Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65, 126–139. <https://doi.org/10.1016/j.compenvurbsys.2017.05.004>
- Open-Meteo. (2024). Historical weather API [Accessed: 2026-05-10].